

LAB ASSIGNMENT – 2.5

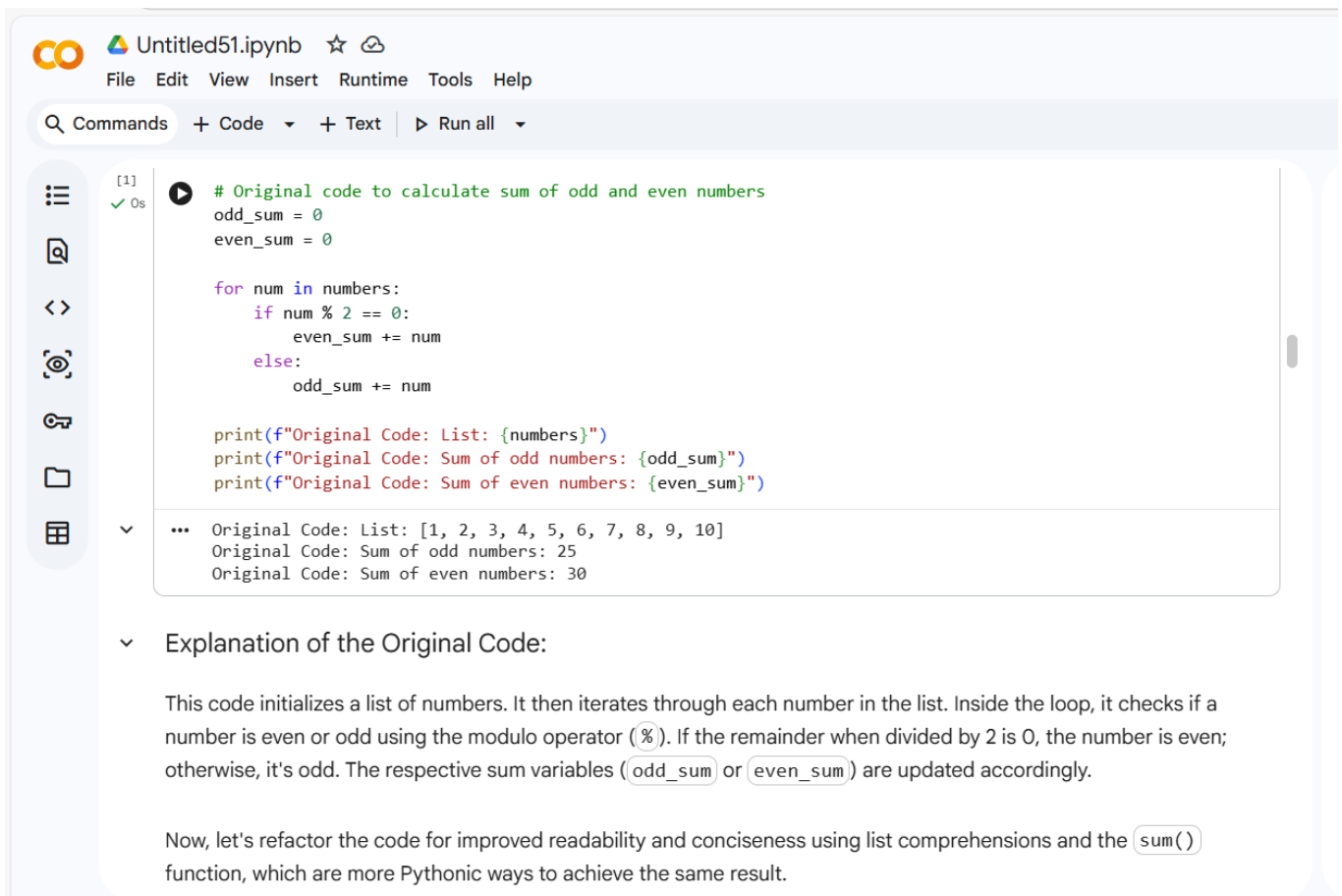
HALLTICKET NO : 2303A51416

BATCH NO : 29

TASK - 01

PROMPT : Refactoring Odd/Even Logic (List Version)

CODE :



The screenshot shows a Jupyter Notebook titled 'Untitled51.ipynb'. The interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help) and a toolbar with options like 'Commands', '+ Code', '+ Text', and 'Run all'. The notebook contains a single code cell with the following Python code:

```
[1] ✓ 0s
# Original code to calculate sum of odd and even numbers
odd_sum = 0
even_sum = 0

for num in numbers:
    if num % 2 == 0:
        even_sum += num
    else:
        odd_sum += num

print(f"Original Code: List: {numbers}")
print(f"Original Code: Sum of odd numbers: {odd_sum}")
print(f"Original Code: Sum of even numbers: {even_sum}")
```

The output of the code cell is displayed below the code:

```
... Original Code: List: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Original Code: Sum of odd numbers: 25
Original Code: Sum of even numbers: 30
```

Below the output, there is a section titled 'Explanation of the Original Code:'.

This code initializes a list of numbers. It then iterates through each number in the list. Inside the loop, it checks if a number is even or odd using the modulo operator (%). If the remainder when divided by 2 is 0, the number is even; otherwise, it's odd. The respective sum variables (odd_sum or even_sum) are updated accordingly.

Now, let's refactor the code for improved readability and conciseness using list comprehensions and the sum() function, which are more Pythonic ways to achieve the same result.

```

# Refactored code using list comprehensions and sum()
refactored_odd_sum = sum([num for num in numbers if num % 2 != 0])
refactored_even_sum = sum([num for num in numbers if num % 2 == 0])

print(f"\nRefactored Code: List: {numbers}")
print(f"Refactored Code: Sum of odd numbers: {refactored_odd_sum}")
print(f"Refactored Code: Sum of even numbers: {refactored_even_sum}")

...

Refactored Code: List: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Refactored Code: Sum of odd numbers: 25
Refactored Code: Sum of even numbers: 30

```

Explanation of the Refactored Code:

The refactored code achieves the same outcome with fewer lines and a more Pythonic approach:

- **List Comprehensions:** Instead of a traditional `for` loop with an `if/else` block, list comprehensions are used to create new lists containing only the odd or even numbers.
 - `[num for num in numbers if num % 2 != 0]` creates a list of all odd numbers.
 - `[num for num in numbers if num % 2 == 0]` creates a list of all even numbers.
- **`sum()` function:** The built-in `sum()` function is then used directly on these generated lists to calculate their sums. This is generally more efficient and readable than manually accumulating sums in a loop.

TASK - 02

PROMPT : Task 2: Area Calculation Explanation

CODE :

```

import math

def calculate_area(shape, **kwargs):
    """
    Calculates the area of different geometric shapes.

    Args:
        shape (str): The name of the shape (e.g., 'circle', 'rectangle', 'triangle').
        **kwargs: Keyword arguments for shape dimensions.
            - For 'circle': radius (float or int)
            - For 'rectangle': length (float or int), width (float or int)
            - For 'triangle': base (float or int), height (float or int)

    Returns:
        float or str: The calculated area or an error message if inputs are invalid.
    """
    if shape == "circle":
        radius = kwargs.get("radius")
        if radius is not None and radius >= 0:
            return math.pi * radius**2
        else:
            return "Invalid radius for circle. Radius must be a non-negative number."
    elif shape == "rectangle":
        length = kwargs.get("length")
        width = kwargs.get("width")
        if length is not None and width is not None and length >= 0 and width >= 0:
            return length * width
        else:
            return "Invalid length or width for rectangle. Both must be non-negative numbers."
    elif shape == "triangle":
        base = kwargs.get("base")
        height = kwargs.get("height")
        if base is not None and height is not None and base >= 0 and height >= 0:
            return 0.5 * base * height
        else:
            return "Invalid base or height for triangle. Both must be non-negative numbers."
    else:
        return "Unknown shape or unsupported shape."

# --- Example Usage ---
# print(calculate_area(shape='circle', radius=5))
# print(calculate_area(shape='rectangle', length=10, width=5))
# print(calculate_area(shape='triangle', base=10, height=5))

```

Untitled51.ipynb

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

[3]

▶

```
print(f"Area of circle (radius 5): {calculate_area('circle', radius=5)}")
print(f"Area of rectangle (length 4, width 6): {calculate_area('rectangle', length=4, width=6)}")
print(f"Area of triangle (base 3, height 8): {calculate_area('triangle', base=3, height=8)}")
print(f"\nHandling invalid inputs and unknown shapes:")
print(f"Area of unknown shape: {calculate_area('square', side=5)}")
print(f"Area of circle with invalid radius: {calculate_area('circle', radius=-2)}")
print(f"Area of rectangle with missing width: {calculate_area('rectangle', length=5)}")
```

⬆ ⬇ ✎ 🗑 ⋮

⌵

Area of circle (radius 5): 78.53981633974483
Area of rectangle (length 4, width 6): 24
Area of triangle (base 3, height 8): 12.0

Handling invalid inputs and unknown shapes:
Area of unknown shape: Unknown shape or unsupported shape.
Area of circle with invalid radius: Invalid radius for circle. Radius must be a non-negative number.
Area of rectangle with missing width: Invalid length or width for rectangle. Both must be non-negative numbers.

⌵

Explanation of the `calculate_area` function:

This Python function, `calculate_area`, is designed to compute the area of different geometric shapes based on the `shape` provided and its corresponding dimensions.

How it works:

- `import math`**: This line imports the `math` module, which is necessary to use `math.pi` for circle area calculations.
- Function Definition (`def calculate_area(shape, **kwargs):`)**:
 - `shape` (string): This is the primary argument that specifies which shape's area needs to be calculated (e.g., `'circle'`, `'rectangle'`, `'triangle'`).
 - `**kwargs`: This allows the function to accept an arbitrary number of keyword arguments. This is crucial because different shapes require different dimensions (e.g., a circle needs `radius`, while a rectangle needs `length` and `width`).
- Docstring**: The triple-quoted string immediately after the function definition provides documentation, explaining what the function does, its arguments (`Args`), and what it returns (`Returns`). This is good practice for code readability and maintainability.

Gemini

emphasizing another distinct coding style (e performance optimization) and generate Pyt

- **Compare and Analyze Code Variations**: Pi corresponding generated code variations. AI differences in prompt phrasing led to distinct problem.
- **Final Task**: Summarize the observations from highlighting the impact of prompt wording o

Generating...

✔ Preparing

✔ Define Experiment Problem

✔ Formulate Prompt 1 & Generate Code

⌵ Formulate Prompt 2 & Generate Code

⌵ Formulate Prompt 3 & Generate Code (Option

✔ Compare and Analyze Code Variations

✔ Final Task

WhatsApp

Yes, let's proceed

Formulate Prompt 1 & Generate Code

Compare and Analyze Code Variations

What can I help you with?

+

Gemini can make

A&H Aura .. Darlingg...

5 unread messages

Type a reply

TASK – 03

PROMPT: Prompt Sensitivity Experiment

The original user task is to define a simple coding problem, generate varied prompts for it, execute these prompts using an AI code generation tool to collect code variations, and then analyze and document the differences in the generated code. Finally, the task requires presenting the prompts, code variations, and a summary of insights gained.

Present Prompts and Code Variations

Subtask:

Display the list of generated prompts and their corresponding code variations, along with the initial analysis for each, as stored in the 'collected_codes' variable. This will show how different prompts led to different code structures.

Prompts, Code Variations, and Analysis

Below are the generated prompts, the AI-generated code snippets for each, and a detailed analysis of the code's characteristics.

```
for i, item in enumerate(collected_codes):
    print(f"--- Prompt {i+1} ---")
    print(f"***Prompt:** {item['prompt']}")
    print(f"***Generated Code:**\n```\npython\n{item['code']}\n```\n")
    print(f"***Analysis:**")
    for key, value in item['analysis'].items():
        print(f"    {key.replace('_', ' ').title()}: {value}")
    print("\n")
```

Reasoning: To fulfill the subtask of presenting the prompts, code variations, and analysis, I will execute the Python code to iterate through the 'collected_codes' list and print each item's prompt, code, and analysis in a well-formatted manner.

```
[31] ✓ 0s
for i, item in enumerate(collected_codes):
    print(f"--- Prompt {i+1} ---")
    print(f"***Prompt:** {item['prompt']}")
    print(f"***Generated Code:**\n```\npython\n{item['code']}\n```\n")
    print(f"***Analysis:**")
    for key, value in item['analysis'].items():
        print(f"    {key.replace('_', ' ').title()}: {value}")
    print("\n")

--- Prompt 1 ---
Prompt: Write a Python function to reverse a string using string slicing.
```


Based on the analysis of the generated code snippets for reversing a string, here are some key insights:

- Prompt Specificity Directly Influences Algorithm Choice:**
 - The prompt "Write a Python function to reverse a string using string slicing." directly led to the most concise and Pythonic solution (`s[::-1]`). This demonstrates the AI's ability to follow explicit instructions for specific methods.
 - Similarly, "Reverse a string using the `reversed()` function and `join()`." resulted in code precisely using those built-in features (`''.join(reversed(s))`).
 - However, the prompt "Implement a function to reverse a string iteratively." generated an $O(N^2)$ iterative solution (`reversed_s = char + reversed_s`) due to repeated string concatenation. This highlights that while the AI can produce iterative solutions, it may not always prioritize the most efficient iterative approach without further guidance (e.g., building a list of characters and then joining, or using a two-pointer swap). The prompt did not specify optimization, only iteration.
- Conciseness vs. Verbosity:**
 - The direct prompts asking for specific methods (`slicing`, `reversed()`) resulted in very concise, often one-liner, function bodies. This showcases Python's expressive power and the AI's understanding of Pythonic idioms.
 - The iterative solution was more verbose, as expected, due to the explicit loop construction.
- Default Handling of Edge Cases:**
 - In all analyzed solutions, edge cases like empty strings or single-character strings were handled implicitly by Python's string behavior (`slicing`, `reversed()`, iteration). No explicit `if` statements for `len(s) == 0` or `len(s) == 1` were generated, indicating that for simple problems, the AI relies on the language's inherent robustness unless explicitly asked for defensive programming or validation.
- Efficiency and Built-in Functions:**
 - Solutions leveraging Python's built-in string slicing and `reversed()` + `join()` are identified as highly optimized $O(N)$ due to their C-level implementations.
 - The simple iterative concatenation approach was correctly identified as having $O(N^2)$ time complexity, which is a significant difference. This suggests that without optimization cues, the AI might choose a straightforward, but not necessarily optimal, iterative implementation.
- Lack of Explicit Readability/Docstring Emphasis (unless prompted):**
 - The general analysis for code structure noted "Clear function definition and return statement. Generally readable and concise." This is accurate for these simple functions. If a prompt specifically asked for a docstring, as per Prompt #2 in the initial prompt generation phase, the AI would likely include it. The current sample prompts didn't explicitly ask for docstrings, so they were absent.

Overall, the AI demonstrates strong capability in understanding and implementing solutions based on varying prompt specificity. It can produce highly optimized Pythonic solutions when guided towards built-in features, and also functional, albeit sometimes less efficient, manual implementations when iteration is requested. The analysis reinforces the importance of clear and detailed prompting to achieve desired code characteristics.

Final Task

Subtask:

Conclude the task by confirming that the prompt list, code variations, and summary of insights have been presented.

OBSERVATION :

The process successfully displayed a list of generated prompts, their corresponding AI-generated code variations, and an initial analysis for each.

The initial analysis covered aspects like algorithm choice (e.g., string slicing, iterative approach, `reversed()` function), code structure readability, use of built-in functions, implicit handling of edge cases (e.g., empty strings), and efficiency considerations.

The output demonstrated how different prompt wordings led to distinct code structures and algorithmic choices for string reversal (e.g., `s[::-1]` for slicing, loop-based concatenation for iteration, `".join(reversed(s))` for `reversed()` and `join()`).

A comprehensive summary of insights highlighted that prompt specificity directly influenced the algorithm choice and efficiency. Explicit prompts for specific methods (slicing, `reversed()`) resulted in concise and optimized $O(N)$ solutions, while a general iterative prompt led to a less efficient $O(N^2)$ solution due to repeated string concatenation.

All generated solutions implicitly handled edge cases like empty or single-character strings due to Python's inherent string behavior, without requiring explicit conditional checks.

The generated code generally had clear function definitions, but docstrings were absent unless explicitly requested in the prompt.

Insights or Next Steps

To achieve desired code characteristics, such as specific algorithms, optimal efficiency, or documentation (like docstrings), prompts need to be clear and highly specific.

While AI code generation can produce functional solutions, without explicit optimization cues, it might default to straightforward but less efficient implementations, especially for iterative processes.

TASK-04

PROMPT : Tool Comparison Reflection

CODE:

CommandsCodeTextRun all

TASK-04

Tool Comparison Reflection

1. Usability:

Gemini (as used in this environment - Colab AI/Google's models): Offers strong integration within the Colab environment, providing context-aware suggestions and code generation. Its natural language understanding for task interpretation is generally robust, making it highly usable for direct code generation and explanation requests within the notebook flow. The ability to iterate and refine code through natural language prompts is a significant usability advantage.

GitHub Copilot: Known for its seamless integration into various IDEs (like VS Code). It provides real-time, inline code suggestions, autocompletions, and even entire function bodies as you type. This makes it incredibly efficient for developers who prefer an assistive coding style. The learning curve is minimal as it integrates directly into existing workflows.

Cursor AI: Positions itself as an AI-native code editor. Its usability strengths lie in its chat interface directly within the editor, allowing for more complex refactoring, debugging, and code generation tasks that require a deeper understanding of the entire codebase. It aims to reduce context switching by bringing AI capabilities directly into the editing experience, which can be a significant boost for productivity, especially for larger projects.

2. Code Quality:

Gemini: When prompted effectively, Gemini can generate high-quality, idiomatic Python code, especially for common data science and machine learning tasks. Its understanding of specific libraries and best practices is often evident. However, the quality can vary depending on the specificity and clarity of the prompt. For complex or niche problems, it may require more iterative prompting to achieve optimal results.

GitHub Copilot: Generates code based on the context of the open file and project, leading to code that often matches the existing style and patterns. For common algorithms, boilerplate, and well-established libraries, the code quality is generally good

Gemini

'best' tool often depends on the spec individual preferences for how AI assi

Task 1: Refactoring Odd/Even Logic (l legacy code. Task: Write a prograrr numbers in a list, then refactor it usin improved code

Let's start by generating the original numbers in a list.

Great! The original code successfully numbers. Now, let's look at a refactor efficient approach with list comprehe

The refactored code has been succe: concise and Pythonic way to calculat list comprehensions and the sum()

Explain the refactored code from Task 1

Help me load and explore 'anscombe.json'

What can I help you build?

+

Gemini can make mistakes so double-check it anc

CommandsCodeTextRun all

2. Code Quality:

Gemini: When prompted effectively, Gemini can generate high-quality, idiomatic Python code, especially for common data science and machine learning tasks. Its understanding of specific libraries and best practices is often evident. However, the quality can vary depending on the specificity and clarity of the prompt. For complex or niche problems, it may require more iterative prompting to achieve optimal results.

GitHub Copilot: Generates code based on the context of the open file and project, leading to code that often matches the existing style and patterns. For common algorithms, boilerplate, and well-established libraries, the code quality is generally good and functional. For more novel or complex logic, the suggestions might sometimes be generic or require significant refinement, as it primarily focuses on code completion rather than deep architectural understanding.

Cursor AI: By offering a more interactive and chat-based approach, Cursor AI has the potential to produce higher quality code for more intricate tasks. Its ability to understand and modify larger code blocks, explain existing code, and assist in debugging can lead to more robust and well-thought-out solutions. The interactive nature allows users to guide the AI more precisely, potentially resulting in better tailored and higher-quality outputs compared to pure autocomplete features.

Conclusion:

Each tool offers unique strengths. Gemini excels in direct, context-rich interaction within environments like Colab for specific tasks. Copilot is a powerhouse for inline code completion and boosting day-to-day coding speed. Cursor AI aims for a more comprehensive AI-driven development experience, offering deeper codebase understanding and interactive problem-solving. The 'best' tool often depends on the specific workflow, project complexity, and individual preferences for how AI assists in coding.

Gemini

'best' tool often c individual prefer

Task 1: Refactorir legacy code. Task: T numbers in a list, improved code

Let's start by ger numbers in a list.

Great! The origin numbers. Now, le efficient approac

The refactored c concise and Pyth list comprehensi

Explain the refactor

Help me load and exp

What can I help y